

PYTHON: basics

- знайомство з рядками

str: створення об'єктів через літерал

упорядковані незмінні послідовності символьних даних

```
>>> string_example_1 = "It is my string 1"  
>>> string_example_2 = 'It is my string 2'  
>>> string_example_3 = "It is string 'with' symbols ' inside"  
>>> string_example_4 = """This is  
...a multiline string""" # for multiline string we can use and single quotes
```

ви вже бачите, що символ в середині рядка може бути символом який відображається або символом який виконує службову функцію - наприклад лапки можуть відображатись або обмежувати рядок. Таких службових символів (або комбінацій символів) - багато. Наприклад - перейти на новий рядок - '\n', горизонтальна табуляція - '\t'. Python коли зустрічає такий символ в рядку - виконує команду якій цей службовий символ відповідає. А якщо ми хочемо просто надрукувати цю послідовність - треба її "екранувати" - поставити перед нею символ "\". Для сприйняття всіх символів рядка як друкуємих - перед рядком можна поставити символ 'r' - від слова "raw" - "сирий".

str: оператори

операція “конкатенації” (склеювання) рядків:

```
>>> string_1 = “first string”  
>>> string_2 = “second string”  
>>> string_1 + string_2  
'first stringsecond string'
```

операція повторення рядків:

```
>>> string_1 * 3  
'first stringfirst stringfirst string'
```

операції порівняння рядків:

```
>>> string_1 == string_2  # однакові символи на однакових позиціях  
False  
>>> string_2 > string_1   # порівнюється довжина  
True
```


str: оператори і вбудовані функції

ми заговорили про “довжину” - поговоримо про вбудовану функцію len():

```
>>> len(string_1) # повертає довжину переданого рядка в символах
12 # увага! що довжина друкується не в лапках (як було раніше)
```

рядок - це впорядкована послідовність односимвольних рядків. Він має довжину. Всі символи кодуються за допомогою спеціальних кодів, ми можемо отримувати ці коди для символів за допомогою вбудованої функції ord() і отримувати символ який відповідає переданому коду за допомогою вбудованої функції chr():

```
>>> ord("a")
97
>>> chr(97)
'a'
```

існує корисний оператор **in** який дає можливість побачити чи є лівий рядок субрядком правого:

```
>>> "python" in "I study python"
True
```

str: індексація

так як рядки - впорядковані послідовності - ми можемо звернутися до складових частин цих послідовностей за допомогою індексів:

```
>>> string_1 = "my string"  
>>> string_1[3]  
's'
```

0	1	2	3	4	5	6	7	8
m	y		s	t	r	i	n	g
-9	-8	-7	-6	-5	-4	-3	-2	-1

кожна складова частина послідовності має “прямий” індекс (з початку до кінця - додатні числа, перше 0) і “зворотній” (з кінця послідовності до початку, перший індекс -1):

```
>>> string_1[-3]  
'i'
```

намагання використати неіснуючий індекс - викликає виключення:

```
>>> string_1[10]  
Traceback (most recent call last):  
  File "<pyshell#27>", line 1, in <module>  
    string_1[10]  
IndexError: string index out of range
```

str: нарізання (slice)

на основі впорядкованості і індексації існує цікава операція - нарізання.
Вона формує новий рядок з елементів існуючого:

```
>>> string_1[1:6]  
'y str'
```

“до останнього вказаного” (не входить в результат)

“з першого вказаного” (входить в результат)

0	1	2	3	4	5	6	7	8
m	y		s	t	r	i	n	g
-9	-8	-7	-6	-5	-4	-3	-2	-1

можна використовувати будь-які індекси (“прямі” і “зворотні”) - і навіть одночасно

```
>>> string_1[-8:6]  
'y str'
```

і навіть не вказувати взагалі перший, останній або два індекса - тоді це розуміється,
відповідно, як “з початку”, “до кінця”, “від початку до кінця”:

```
>>> string_1[:6]  
'my str'  
>>> string_1[-8:]  
'y string'
```


str: нарізання (slice)

оператор нарізання може прийняти третій параметр - крок:

```
>>> string_1[-8:7:2]  
'ysr'
```

якщо крок буде від'ємним - то ми можемо змінити черговість приєднання символів з рядка-джерела до рядка-результату:

```
>>> string_1[7:1:-2]  
'nrs'
```

це часто використовується для отримання “реверсованого” рядка:

```
>>> string_1[::-1]  
'gnirts ym'
```

0	1	2	3	4	5	6	7	8
m	y		s	t	r	i	n	g
-9	-8	-7	-6	-5	-4	-3	-2	-1

str: форматування

так як рядки ми часто використовуємо для друку інформації яку читають інші - в python існує декілька шляхів форматування виводу. Ми познайомимось з одним з них - f-рядки.

```
>>> name = "Maria"
>>> position = "python developer"
>>> number_months_in_company = 18
>>> print(f"{name} works in company {number_months_in_company} monts on position {position}.")
Maria works in company 18 monts on position python developer.
```

у фігурних лапках f-рядків можуть використовуватись не лише змінні і об'єкти, а й операції над ними і виклики функцій:

```
>>> x = 10
>>> y = 25
>>> print(f"{x} + {y} = {x + y}")
10 + 25 = 35
>>> print(f"the string 'my string' consists of {len('my string')} characters")
the string 'my string' consists of 9 character
```


str: вбудовані рядкові методи

“методами” називають функції, які викликаються від імені об’єктів і обробляють, як правило, саме ці об’єкти. Наприклад - рядки мають методи `str.upper()` та `str.lower()` які повертають новий рядок, який відповідає базовому але складається виключно з великих або маленьких символів відповідно:

```
>>> "MyAwesomeString".upper()  
'MYAWESOMESTRING'  
>>> "MyAwesomeString".lower()  
'myawesomestring'
```

зверніть увагу, що використання методу не змінює базовий рядок, а повертає новий. Рядки - незмінні і не можуть бути змінені протягом свого життєвого циклу:

```
>>> company_name = "grammarly"  
>>> company_name.capitalize()  
'Grammarly'  
>>> company_name  
'grammarly'
```

str: summary

- рядки (**str**) - впорядковані незмінні послідовності символів (фактично - односимвольних рядків). Створюються за допомогою лапок.
- у складі рядків бувають символи які відображаються і службові. Для зміни режиму символа може використовуватись символ "\", або використання "сирого" рядка за допомогою "r" перед лапками при визначенні рядка
- підтримують логічно зрозумілі оператори "+" (конкатенація), "*" (повторення), "in" (входження), порівняння ("лексикографічний порядок")
- існують функції які можуть у якості аргументів приймати рядки. Ми познайомились з функціями len(str) - повертає довжину рядка, ord(symbol) - повертає код (ціле число) який відповідає символу, chr(number) - повертає символ який відповідає коду number
- завдяки впорядкованості - ми можемо звернутись до елементів рядка використовуючи квадратні дужки - str[number]. Існують дві системи індексації.
- для рядків існує операція "нарізання" - slice - яка за допомогою синтаксису квадратних дужок і двох - str[start, stop], або трьох параметрів str[start, stop, step] дозволяють отримати новий рядок який будується з символів базового.
- ми познайомились з f-рядками, що допомагає при формуванні рядка використовувати раніше визначені об'єкти для підстановки.
- рядкові методи - спеціальні "функції" які можуть виконати певні дії з конкретним рядком. Викликаються за допомогою крапкової нотації.

Самостійне опрацювання:

- в стандартній REPL Python використайте функцію `help(obj)` для отримання підказки по типу `str`
- уважно прочитайте опис дій “звичайних” методів (імена яких починається з букви, а не з символу підкреслення). Не треба запам’ятовувати деталі, треба ознайомитись з існуючими.
- в тій же REPL створіть рядок (“I am learning Python”) і за допомогою вбудованих методів:
 - отримайте з нього рядок, що складається з великих (маленьких) символів
 - отримуйте рядок в якому базовий рядок буде відцентровано в рядку з 100 символів з заповненням зліва і справа - пробілами, знаками “-”
 - отримайте кількість в рядку символів “n”, пробілів.
 - * (ускладнене) - за допомогою вбудованих методів отримайте з базового рядок в якому всі пробіли замінені на символ “+”, символ “e” -> “a”, “a” -> “e”

Наступний матеріал: тип даних *bool*.